CHAPTER 6

Neo4J

A bungee cord is a helpful tool because you can use it to tie together the most disparate of things, no matter how awkwardly shaped or ill fitting they may be. In a lot of ways, Neo4j is the bungee cord of databases, a system intended not so much to store information about things as to *tie them together and record their connections with each other*.

Neo4j is a member of a family of databases known as *graph databases* because it stores data as a graph (in the mathematical sense). Neo4j is known for being "whiteboard friendly" because virtually any diagram that you can draw using boxes and lines on a whiteboard can be stored in Neo4j.

Neo4j focuses more on the *relationships between* values than on the *commonalities among sets of* values (such as collections of documents or tables of rows). In this way, it can store highly variable data in a natural and straightforward way.

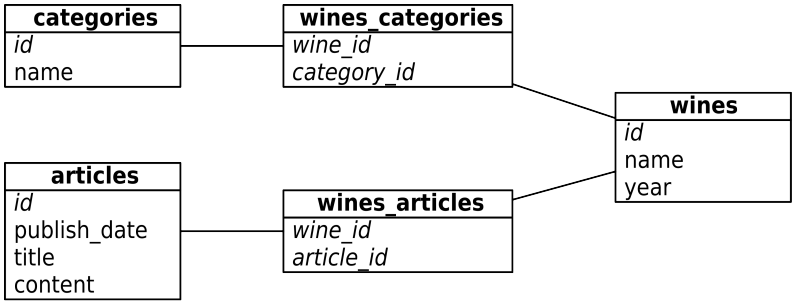
On one side of the scale spectrum, Neo4j is small enough to be embedded into nearly any application; on the other side of the spectrum, Neo4j can run in large clusters of servers using master-slave replication and store tens of billions of nodes and as many relationships. In other words, Neo4j can handle just about any size problem that you can throw at it.

Neo4j Is Whiteboard Friendly

Imagine you need to create a wine suggestion engine in which wines are categorized by different varieties, regions, wineries, vintages, and designations. Imagine that you also need to keep track of things like articles describing those wines written by various authors and to enable users to track their favorite wines.

If you were using a relational model, you might create a category table and a many-to-many relationship between a single winery's wine and some

combination of categories and other data. But this isn't quite how humans mentally model data. In the following figure, compare this wine suggestion schema in relational UML:



to this wine suggestion data on a whiteboard:

There's an old saying in the relational database world: *on a long enough timeline, all fields become optional*. Neo4j handles this implicitly by providing values and structure only where necessary. If a wine blend has no vintage, add a bottle year and point the vintages to the blend node instead. In graph databases such as Neo4j there is simply no schema to adjust.

Over the next three days you'll learn how to interact with Neo4j through a web console, using a querying language called Cypher, then via a REST interface, and finally through search indexes. You'll work with some simple graphs as well as some larger graphs with graph algorithms. Finally, on Day 3, you'll take a peek at the enterprise tools that Neo4j provides for mission-critical applications, from full ACID-compliant transactions to high-availability clustering and incremental backups.

In this chapter, we'll use the Neo4j 3.1.4 Enterprise Edition. Most of the actions you perform on Days 1 and 2 can actually use the GPL Community edition, but we'll require some enterprise functionality for Day 3: Distributed High Availability. You can download a free trial version of the Enterprise Edition from the Neo4j website.

Day 1: Graphs, Cypher, and CRUD

Today we're really going to jump in with both feet. In addition to exploring the Neo4j web interface, we'll get deep into graph database terminology and CRUD. Much of today will be learning how to query a graph using a querying language called *Cypher*. The concepts here differ significantly from other databases we've looked at so far, which have largely taken a document- or record-based view of the world. In Neo4j, nodes inside of graphs act like documents because they store properties, but what makes Neo4j special is that the *relationship* between those nodes takes center stage.

But before we get to all that, let's start with the web interface to see how Neo4j represents data in graph form and how to navigate that graph. After you've downloaded and unzipped the Neo4j package, `cd` into the Neo4j directory and start up the server like this:

```
$ bin/neo4j start
```

To make sure you're up and running, try `curl`ing this URL:

```
$ curl http://localhost:7474/db/data/
```

Like CouchDB, the default Neo4j package comes equipped with a fully featured web administration tool and data browser, which is excellent for experimentation. Even better, it has one of the coolest graph data browsers we've ever seen. This is perfect for getting started because graph traversal can feel very awkward at first try.

Neo4j's Web Interface

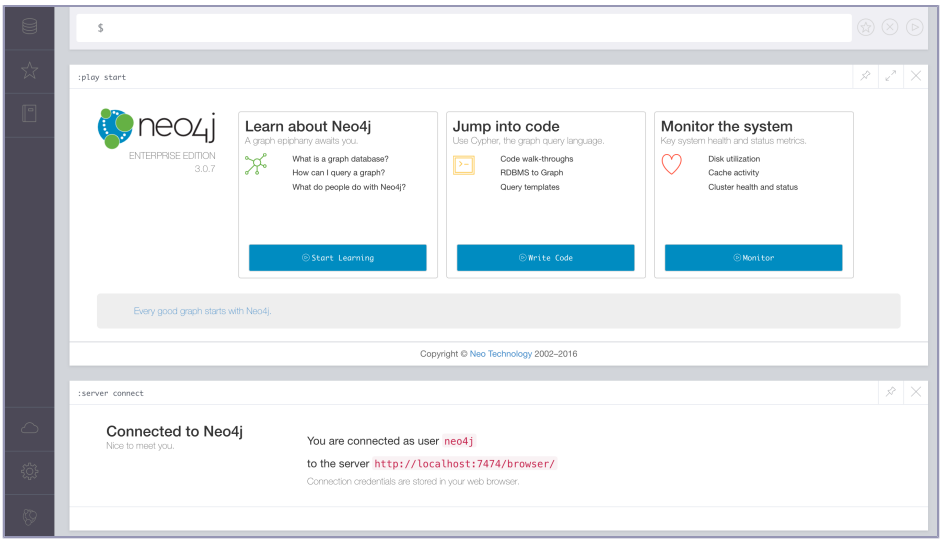
Launch a web browser and navigate to the administration page.[1]

You'll be greeted by a colorful dashboard like the one in the figure on page 180.

In the Connect to Neo4j component, sign in using the default username and password (enter `neo4j` for both). That will open up a command-line-style interface at the top of the page (distinguished by the `$` on the far left). Type in `:server connect` to connect to the database.

You can enter `:help commands` at any time for an in-depth explanation of the existing commands. `:help cypher` will bring up a help page with instructions for specific Cypher commands (more on Cypher, the querying language we'll be using through this web interface, in a moment).

1. http://localhost:7474/browser/



Nodes and Relationships: A Note on Terminology

A *node* in a graph database is not entirely unlike the nodes we talked about in prior chapters. Previously, when we spoke of a node, we meant a physical server in a network. If you viewed the entire network as a huge interconnected graph, a server node was a point, or *vertex*, between the server *relationships*, or *edges*.

In Neo4j, a node is conceptually similar: It's a vertex between edges that may hold data. That data is stored as a set of key-value pairs (as in many other non-relational databases we've talked about).

Neo4j via Cypher

There are several ways that you can interact with Neo4j. In addition to client libraries in a wide variety of programming languages (as with the other databases in this book), you can also interact with Neo4j via a REST API (more on this in Day 2), and via two querying languages created with Neo4j exclusively in mind: *Gremlin* and *Cypher*. While Gremlin has some interesting properties, Cypher is now considered standard.

Cypher is a rich, Neo4j-specific graph traversal language. In Cypher, as in mathematical graph theory, graph data points are called *nodes*. Unlike in graph theory, however, graphs in Cypher consist of *nodes* rather than *vertices* (as they are called in graph theory) and connections between nodes are called *relationships* (rather than *edges*). Statements used to query Neo4j graphs in Cypher typically look something like this:

```
$ MATCH [some set of nodes and/or relationships]
  WHERE [some set of properties holds]
  RETURN [some set of results captured by the MATCH and WHERE clauses]
```

In addition to querying the graph using MATCH, you can create new nodes and relationships using CREATE, update the values associated with nodes and relationships using UPDATE, and much more. That's fairly abstract, but don't worry—you'll get the hang of it via examples over the course of the next few sections.

At the moment, our not-so-exciting Neo4j graph consists of no nodes and no relationships. Let's get our hands dirty and change that by adding a node for a specific wine to our graph. That node will have a few properties: a name property with a value of Prancing Wolf, a style property of ice wine, and a vintage property of 2015. To create this node, enter this Cypher statement into the console:

```
$ CREATE (w:Wine {name:"Prancing Wolf", style: "ice wine", vintage: 2015})
```

In the section of the web UI immediately below the console, you should see output like that in the figure that follows.

At the top, you'll see the Cypher statement you just ran. The Rows tile shows you the nodes and/or relationships that you created in the last Cypher statement, and the Code tile provides in-depth information about the action you just completed (mostly info about the transaction that was made via Neo4j's REST API).

At any time, we can access all nodes in the graph, kind of like a SELECT * FROM entire_graph statement:

```
$ MATCH (n)
  RETURN n;
```

At this point, that will return just one solitary node. Let's add some others. Remember that we also want to keep track of wine-reviewing publications in our graph. So let's create a node representing the publication *Wine Expert Monthly*:

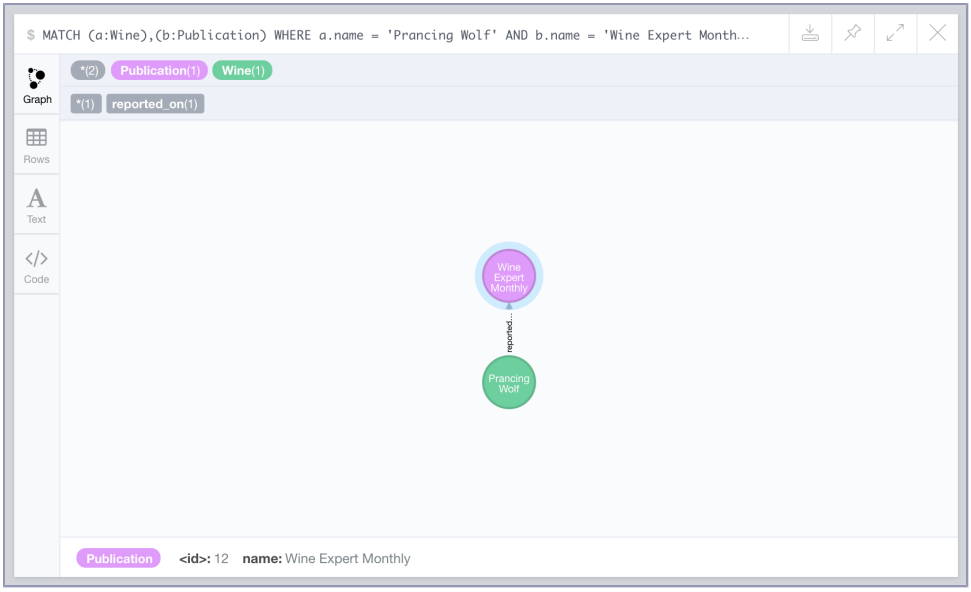
```
$ CREATE (p:Publication {name: "Wine Expert Monthly"})
```

In the last two statements, Wine and Publication were *labels* applied to the nodes, not types. We could create a node with the label Wine that had a completely different set of properties. Labels are extremely useful for querying purposes, as you'll see in a bit, but Neo4j doesn't require you to have predefined types. If you *do* want to enforce types, you'll have to do that at the application level.

So now we have a graph containing two nodes but they currently have no relationship with one another. Because *Wine Expert Monthly reports on* this Prancing Wolf wine, let's create a reported_on relationship that connects the two nodes:

```
$ MATCH (p:Publication {name: "Wine Expert Monthly"}),
    (w:Wine {name: "Prancing Wolf", vintage: 2015})
  CREATE (p)-[r:reported_on]->(w)
```

In this statement, we've MATCHed the two nodes that we want to connect via their labels (Wine and Publication) and their name property, created a reported_on relationship and stored that in the variable r, and finally RETURNed that relationship. You can see the end result in the figure that follows.



If you click on the relationship between the nodes in the web UI, you can see the ID of the relationship is 0. You can use Neo4j's REST interface to access information about the relationship at http://localhost:7474/db/data/relationship/0 or via Cypher by running:

```
$ MATCH ()-[r]-()
  WHERE id(r) = 0
  RETURN r
```

Relationships, like nodes, can contain properties and can be thought of as objects in their own right. After all, we don't want to know simply *that* a relationship exists; we want to know *what* constitutes that relationship. Let's say that we want to specify which score *Wine Expert Monthly* gave the Prancing Wolf wine. We can do that by adding a rating property to the relationship that we just created.

```
$ MATCH ()-[r]-()
  WHERE id(r) = 0
  SET r.rating = 97
  RETURN r
```

We also could've specified the rating when creating the relationship, like this:

```
$ MATCH (p:Publication {name: "Wine Expert Monthly"}),
    (w:Wine {name: "Prancing Wolf"})
  CREATE (p)-[r:reported_on {rating: 97}]->(w)
```

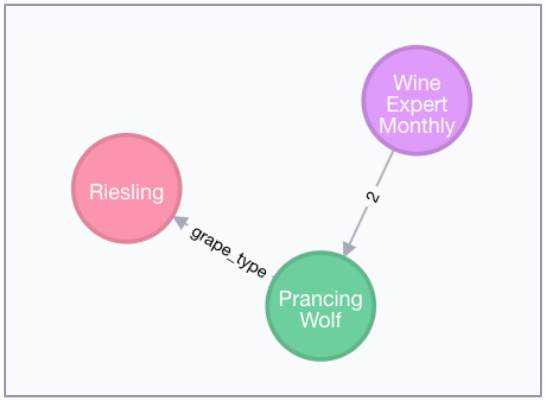
At this point, if you display the entire graph again using MATCH (n) RETURN n; and click on the relationship, you'll see that rating: 97 is now a property of the reported_on relationship. Another bit of info that we want to note is that the Prancing Wolf wine is made from the *Riesling* grape. We *could* insert this info by adding a grape_type: Riesling property to the Prancing Wolf node, but let's do things in a more Neo4j-native fashion instead by creating a new node for the Riesling grape type and adding relationships to wines of that type:

```
$ CREATE (g:GrapeType {name: "Riesling"})
```

Let's add a relationship between the Riesling node and the Prancing Wolf node using the same method:

```
$ MATCH (w:Wine {name: "Prancing Wolf"}),(g:GrapeType {name: "Riesling"})
  CREATE (w)-[r:grape_type]->(g)
```

Now we have a three-node graph: a wine, a type of grape, and a publication.



So far, we've created and updated both nodes and relationships. You can also delete both from a graph. The following are three Cypher statements that will create a new node, establish a relationship between that node and one of our existing nodes, delete the relationship, and then delete the node (you can't delete a node that still has relationships associated with it):

```
$ CREATE (e: EphemeralNode {name: "short lived"})
$ MATCH (w:Wine {name: "Prancing Wolf"}),
    (e:EphemeralNode {name: "short lived"})
  CREATE (w)-[r:short_lived_relationship]->(e)
$ MATCH ()-[r:short_lived_relationship]-()
  DELETE r
$ MATCH (e:EphemeralNode)
  DELETE e
```

Our wine graph is now back to where it was before creating the short lived node. Speaking of deletion, if you ever want to burn it all down and start from scratch with an empty graph, you can use the following command at any time to delete all nodes and relationships. But beware! This command will delete the entire graph that you're working with, so run it only if you're sure that you're ready to move on from a graph's worth of data for good.

```
$ MATCH (n)
  OPTIONAL MATCH (n)-[r]-()
  DELETE n, r
```

Now that you know how to start from scratch, let's continue building out our wine graph. Wineries typically produce more than one wine. To express that relationship in an RDBMS, we might create a separate table for each winery and store wines that they produce as rows. The most natural way to express this in Neo4j would be—you guessed it—to represent wineries as nodes in the graph and create relationships between wineries and wines. Let's create a node for Prancing Wolf Winery and add a relationship with the Prancing Wolf wine node that we created earlier:

```
$ CREATE (wr:Winery {name: "Prancing Wolf Winery"})
$ MATCH (w:Wine {name: "Prancing Wolf"}),
    (wr:Winery {name: "Prancing Wolf Winery"})
  CREATE (wr)-[r:produced]->(w)
```

We'll also add two more wines produced by Prancing Wolf Winery—a Kabinett and a Spätlese—and also create produced relationships and specify that all of the Prancing Wolf wines are Rieslings.

```
$ CREATE (w:Wine {name:"Prancing Wolf", style: "Kabinett", vintage: 2002})
$ CREATE (w:Wine {name: "Prancing Wolf", style: "Spätlese", vintage: 2010})
$ MATCH (wr:Winery {name: "Prancing Wolf"}),(w:Wine {name: "Prancing Wolf"})
  CREATE (wr)-[r:produced]->(w)
$ MATCH (w:Wine),(g:GrapeType {name: "Riesling"})
  CREATE (w)-[r:grape_type]->(g)
```

This will result in a graph that's fully fleshed out, like the one shown in the figure that follows.

Schemaless Social

In addition to knowing about wines, wineries, and publications, we want our wine graph to have a social component—that is, we want to know about the *people* affiliated with these wines and their relationships with one another. To do that, we just need to add more nodes. Suppose that you want to add three people, two who know each other and one stranger, each with their own wine preferences.

Alice has a bit of a sweet tooth so she's a big fan of ice wine.

```
$ CREATE (p:Person {name: "Alice"})
$ MATCH (p:Person {name: "Alice"}),
    (w:Wine {name: "Prancing Wolf", style: "ice wine"})
  CREATE (p)-[r:likes]->(w)
```

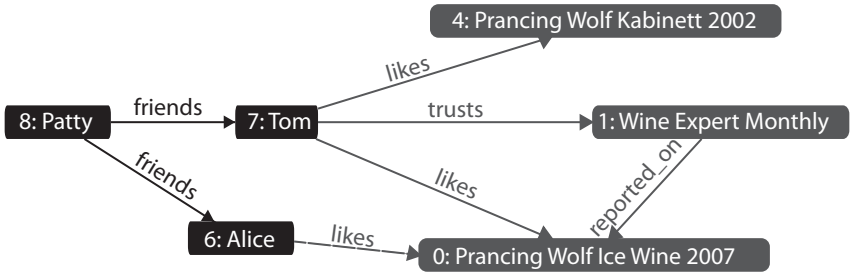
Tom likes Kabinett and ice wine and trusts anything written by *Wine Expert Monthly*.

```
$ CREATE (p: Person {name: "Tom"})
$ MATCH (p:Person {name: "Tom"}),
    (w:Wine {name: "Prancing Wolf", style: "ice wine"})
  CREATE (p)-[r:likes]->(w)
$ MATCH (p:Person {name: "Tom"}),
    (pub:Publication {name: "Wine Expert Monthly"})
  CREATE (p)-[r:trusts]->(pub)
```

Patty is friends with both Tom and Alice but is new to wine and has yet to choose any favorites.

```
$ CREATE (p:Person {name: "Patty"})
$ MATCH (p1:Person {name: "Patty"}),
    (p2:Person {name: "Tom"})
  CREATE (p1)-[r:friends]->(p2)
$ MATCH (p1:Person {name: "Patty"}),
    (p2:Person {name: "Alice"})
  CREATE (p1)-[r:friends]->(p2)
```

Note that without changing any fundamental structure of our existing graph, we were able to superimpose behavior beyond our original intent. The new nodes are related, as you can see in the following figure.



Stepping Stones

Thus far, we've mostly been performing simple, almost CRUD-like operations using Cypher. You can do *a lot* with these simple commands, but let's dive in and see what else Cypher has to offer. First, let's explore Cypher's syntax for querying all relationships that a node has with a specific type of node. The --> operator lets us do that. First, let's see all nodes associated with Alice:

```
$ MATCH (p:Person {name: "Alice"})-->(n)
  RETURN n;
```

Now let's see all of the people that Alice is friends with, except let's return only the `name` property of those nodes:

```
$ MATCH (p:Person {name: "Alice"})-->(other: Person)
  RETURN other.name;
```

That should result in two returned values: `Patty` and `Tom`. Now let's say that we want to see which nodes with the label `Person` are in the graph, but excluding Patty (boo, Patty!). Note the <> operator, which is used instead of != in Cypher:

```
$ MATCH (p:Person)
  WHERE p.name <> 'Patty'
  RETURN p;
```

Thus far, all of our queries have sought out nodes adjacent to one another. But we also said at the beginning of the chapter that Neo4j is an extremely scalable database capable of storing tons of nodes and relationships. Cypher is absolutely up to the task of dealing with far more complex relationships than the ones we've seen thus far. Let's add some nodes that aren't directly related to Patty (for Alice's friend Ahmed and Tom's friend Kofi) and then query for a relationship.

```
$ CREATE (p1:Person {name: "Ahmed"}), (p2:Person {name: "Kofi"});
$ MATCH (p1:Person {name: "Ahmed"}),(p2:Person {name: "Alice"})
  CREATE (p1)-[r:friends]->(p2);
$ MATCH (p1:Person {name: "Kofi"}),(p2:Person {name: "Tom"});
  CREATE (p1)-[r:friends]->(p2);
```

Cypher lets us query for friends of friends of Alice like this:

```
$ MATCH
    (fof:Person)-[:friends]-(f:Person)-[:friends]-(p:Person {name: "Patty"})
  RETURN fof.name;
```

As expected, this returns two values: Ahmed and Kofi.

Indexes, Constraints, and "Schemas" in Cypher

Neo4j doesn't enable you to enforce hard schemas the way that relational databases do, but it does enable you to provide *some* structure to nodes in your graphs by creating indexes and constraints for specified labels.

As with many other databases in this book, you can provide a nice speed-up for computationally expensive queries by creating indexes on labels and properties associated with that label. Remember that each Wine in our graph has a name property. You can create an index on that type/property combination like this:

```
$ CREATE INDEX ON :Wine(name);
```

You can easily remove indexes at any time:

```
$ DROP INDEX ON :Wine(name);
```

Indexes are super easy to use in Neo4j because you don't really have to *do* much to use them. Once you've established an index for nodes with a specific label and property, you can continue to query those nodes as you did before, and Neo4j will figure out the rest. This query, which returns all nodes with the label Wine, would look exactly the same before and after creating an index on Wine/name:

```
$ MATCH (w:Wine {name: 'Some Name'})
  RETURN w;
```

While indexes can help speed up queries, *constraints* can help you sanitize your data inputs by preventing writes that don't satisfy criteria that you specify. If you wanted to ensure that every Wine node in your graph had a unique name, for example, you could create this constraint:

```
$ CREATE CONSTRAINT ON (w:Wine) ASSERT w.name IS UNIQUE;
```

Functions in Cypher

In the Postgres chapter, we covered stored procedures, which enable you to write functions *in Postgres*—and highly complex ones at that—that can be used to process data. Stored procedures shift responsibility for data processing to the database rather than keeping it in the application, which is sometimes a good idea, sometimes not.

Neo4j offers a wide variety of Cypher *functions* that work just liked stored procedures. These built-in functions can be used to manipulate strings and other types, calculate spatial distances, perform mathematical calculations, and much more. Using these functions can help you maintain a nice balance between in-database and application-side data crunching.

Check out the Cypher documentation for a full listing.[a] You'll find functions such as `filter()` for applying complex filtering operations, `abs()` for calculating the absolute value of integers, `range()` for generating a range of integers, and more. You can also write your own functions in Java and call those functions from Cypher, but that won't be covered in this book.

a. http://neo4j.com/docs/developer-manual/current/cypher/functions/

Now, if you try to create two `Wine` nodes with the same name, you'll get an error:

```
$ CREATE (w:Wine {name: "Daring Goat", style: "Spätlese", vintage: 2008});
$ CREATE (w:Wine {name: "Daring Goat", style: "Riesling", vintage: 2006});
WARNING: Node 219904 already exists...
```

Even better, when you create a constraint, Neo4j will automatically check your existing data to make sure that all nodes with the given label conform to the constraint. Like indexes, constraints can be removed using a `DROP` statement, though make sure to include the entire constraint statement:

```
$ DROP CONSTRAINT ON (w:Wine) ASSERT w.name IS UNIQUE;
```

Keep in mind that you cannot apply a constraint to a label that already has an index, and if you do create a constraint on a specific label/property pair, an index will be created automatically. So *usually* you'll only need to explicitly create a constraint *or* an index.

If you want to see the status of a label's "schema," you can see that information in the shell:

```
$ schema ls -l :Wine
Indexes
  ON :Wine(name) ONLINE (for uniqueness constraint)

Constraints
  ON (wine:Wine) ASSERT wine.name IS UNIQUE
```

Although Neo4j isn't fundamentally schema-driven the way that relational databases are, indexes and constraints will help keep your queries nice and fast and your graph sane. They are an absolute must if you want to run Neo4j in production.

Day 1 Wrap-Up

Today we began digging into the graph database Neo4j and the Cyper querying language—and what a different beast we've encountered! Although we didn't cover specific design patterns per se, our brains are now buzzing with the strange and beautiful possibilities opened up by the graph database worldview. Remember that if you can draw it on a whiteboard, you can store it in a graph database.

Day 1 Homework

Find

1. Browse through the Neo4j docs at https://neo4j.com/docs and read more about Cypher syntax. Find some Cypher features mentioned in those docs that we didn't have a chance to use here and pick your favorite.

2. Experiment with an example graph consisting of movie-related data by going back to the browser console at http://localhost:7474/browser, typing `:play movie-graph` into the web console, and following the instructions.

Do

1. Create a simple graph describing some of your closest friends, your relationships with them, and even some relationships between your friends. Start with three nodes, including one for yourself, and create five relationships.

Day 2: REST, Indexes, and Algorithms

Today we'll start with Neo4j's REST interface. First, we'll use the REST interface to create and index nodes and relationships, and to execute full-text search. Then we'll look at a plugin that lets us execute Cypher queries on the server via REST, freeing our code from the confines of the web console.

Taking a REST

Just like HBase, Mongo, and CouchDB, Neo4j ships with a REST interface. One reason all of these databases support REST is that it allows language-agnostic interactions in a standard connection interface. You can connect to

Neo4j—which requires Java to work—from a separate machine that isn't running Java at all.

Before beginning today's exercises, check to make sure that the REST server is running by issuing a curl command to the base URL. It runs on the same port as the web admin tool you used yesterday, at the /db/data/ path (note the slash at the end).

```
$ curl http://localhost:7474/db/data/
{
  "extensions" : { },
  "node" : "http://localhost:7474/db/data/node",
  "relationship" : "http://localhost:7474/db/data/relationship",
  "node_index" : "http://localhost:7474/db/data/index/node",
  "relationship_index" : "http://localhost:7474/db/data/index/relationship",
  "extensions_info" : "http://localhost:7474/db/data/ext",
  "relationship_types" : "http://localhost:7474/db/data/relationship/types",
  "batch" : "http://localhost:7474/db/data/batch",
  "cypher" : "http://localhost:7474/db/data/cypher",
  "indexes" : "http://localhost:7474/db/data/schema/index",
  "constraints" : "http://localhost:7474/db/data/schema/constraint",
  "transaction" : "http://localhost:7474/db/data/transaction",
  "node_labels" : "http://localhost:7474/db/data/labels",
  "neo4j_version" : "3.0.7"
}
```

If the server is running, this request will return a nice JSON object describing the URLs that you can use for other commands, such as node-, relationship-, and index-related actions.

Creating Nodes and Relationships Using REST

It's as easy to create nodes and relationships over REST in Neo4j as in CouchDB or Mongo. Creating a node requires a POST to the /db/data/node path with JSON data. As a matter of convention, it pays to give each node a name property. This makes viewing any node's information easy: just call name.

```
$ curl -i -XPOST http://localhost:7474/db/data/node \
  -H "Content-Type: application/json" \
  -d '{
    "name": "P.G. Wodehouse"
    "genre": "British Humour"
  }'
```

When posted, you'll get the node path in the header and a body of metadata about the node (both are truncated here for brevity). All of this data is retrievable by calling GET on the given header Location value (or the self property in the metadata).

```
HTTP/1.1 201 Created
Location: http://localhost:7474/db/data/node/341
Content-Type: application/json; charset=UTF-8

{
  "extensions" : { },
  "metadata" : {
    "id" : 341,
    "labels" : [ ]
  },
  "paged_traverse" : "...",
  "outgoing_relationships": "...",
  "data" : {
    "name" : "P.G. Wodehouse",
    "genre" : "British Humour"
  }
}
```

If you just want to fetch the node properties (not the metadata), you can GET that by appending /properties to the node URL or even an individual property by further appending the property name.

```
$
$ curl http://localhost:7474/db/data/node/9/properties/genre
"British Humour"
```

One node doesn't do us much good, so go ahead and create another one with these properties: ["name" : "Jeeves Takes Charge", "style" : "short story"].

Because P. G. Wodehouse wrote the short story "Jeeves Takes Charge," we can make a relationship between them.

```
$ curl -i -XPOST http://localhost:7474/db/data/node/9/relationships \
  -H "Content-Type: application/json" \
  -d '{
    "to": "http://localhost:7474/db/data/node/10",
    "type": "WROTE",
    "data": {"published": "November 28, 1916"}
  }'
```

A nice thing about the REST interface is that it actually reported on how to create a relationship early in the body metadata's create_relationship property. In this way, the REST interfaces tend to be mutually discoverable.

Finding Your Path

Through the REST interface, you can find the path between two nodes by posting the request data to the starting node's /paths URL. The POST request data must be a JSON string denoting the node you want the path to, the type of relationships you want to follow, and the path-finding algorithm to use.

For example, we're looking for a path following relationships of the type WROTE from node 1 using the shortestPath algorithm and capping out at a depth of 10.

```
$ curl -X POST http://localhost:7474/db/data/node/9/paths \
  -H "Content-Type: application/json" \
  -d '{
    "to": "http://localhost:7474/db/data/node/10",
    "relationships": {"type": "WROTE"},
    "algorithm": "shortestPath",
    "max_depth": 10
  }'
[ {
  "start" : "http://localhost:7474/db/data/node/9",
  "nodes" : [
    "http://localhost:7474/db/data/node/9",
    "http://localhost:7474/db/data/node/10"
  ],
  "length" : 1,
  "relationships" : [ "http://localhost:7474/db/data/relationship/14" ],
  "end" : "http://localhost:7474/db/data/node/10"
} ]
```

The other path algorithm choices are allPaths, allSimplePaths, and dijkstra. You can find information on these algorithms in the online documentation,[2] but detailed coverage is outside the scope of this book.

Indexing

Like other databases we've seen, Neo4j supports fast data lookups by constructing indexes. We briefly mentioned indexing toward the end of Day 1. There is a twist, though. Unlike other database indexes where you perform queries in much the same way as without one, Neo4j indexes have a different path because the indexing service is actually a separate service.

The simplest index is the key-value or hash style. You key the index by some node data, and the value is a REST URL, which points to the node in the graph. You can have as many indexes as you like, so we'll name this one authors. The end of the URL will contain the author name we want to index and pass in node 1 as the value (or whatever your Wodehouse node was).

```
$ curl -X POST http://localhost:7474/db/data/index/node/authors \
  -H "Content-Type: application/json" \
  -d '{
    "uri": "http://localhost:7474/db/data/node/9",
    "key": "name",
    "value": "P.G.+Wodehouse"
  }'
```

2. https://neo4j.com/blog/graph-search-algorithm-basics/

Retrieving the node is simply a call to the index, which you'll notice doesn't return the URL we specified but rather the actual node data.

```
$ curl http://localhost:7474/db/data/index/node/authors/name/P.G.+Wodehouse
```

Besides key-value, Neo4j provides a full-text search inverted index, so you can perform queries like this: "Give me all books that have names beginning with Jeeves." To build this index, you need to build it against the entire dataset, rather than our one-offs earlier. Neo4j incorporates Lucene to build our inverted index.

```
$ curl -X POST http://localhost:7474/db/data/index/node \
  -H "Content-Type: application/json" \
  -d '{
    "name": "fulltext",
    "config": {"type": "fulltext", "provider": "lucene"}
    }'
```

The POST will return a JSON response containing information about the newly added index.

```
{
  "template": "http://localhost:7474/db/data/index/...",
  "provider": "lucene",
  "type": "fulltext"
}
```

Now if you add Wodehouse to the full-text index, you get this:

```
$ curl -X POST http://localhost:7474/db/data/index/node/fulltext \
  -H "Content-Type: application/json" \
  -d '{
    "uri": "http://localhost:7474/db/data/node/9",
    "key": "name",
    "value" : "P.G.+Wodehouse"
  }'
```

Then a search is as easy as a Lucene syntax query on the index URL.

```
$ curl http://localhost:7474/db/data/index/node/fulltext?query=name:P*
```

Indexes can also be built on edges like we did previously; you just have to replace the instances of *node* in the URLs with *relationship*—for example, http://localhost:7474/db/data/index/relationship/published/date/1916-11-28.

REST and Cypher

We spent much of Day 1 using Cypher and the first half of today using the REST interface. If you wondered which you should use, fear not. The Neo4j REST interface has a Cypher plugin (which is installed by default in the version

of Neo4j we're using).[3] You can send through REST any commands you could in the Cypher console. This allows you the power and flexibility of both tools in production. This is a great combination because Cypher is better geared toward powerful queries, where REST is geared toward deployment and language flexibility.

The following code will return the names of all relationships. You only need to send the data to the plugin URL as a JSON string value, under the field query.

```
$ curl -X POST \
  http://localhost:7474/db/data/cypher \
  -H "Content-Type: application/json" \
  -d '{
    "query": "MATCH ()-[r]-() RETURN r;"
  }'
{
  "columns" : [ "n.name" ],
  "data" : [ [ "Prancing Wolf" ], [ "P.G. Wodehouse" ] ]
}
```

From here on out, code samples will use Cypher (as on Day 1) because it has a much more clean and compact syntax. The REST interface is a good thing to bear in mind, though, for use cases in which it would be beneficial to set up an HTTP client to fetch information from Neo4j.

Big Data

Up until now, we've dealt with very small datasets, so now it's time to see what Neo4j can do with some big data. We'll explore a dataset covering information about over 12,000 movies and over 40,000 actors, 6,000 directors, and others involved in those movies.

This dataset has been made available[4] by the good folks at Neo4j, who have conveniently made the data directly digestible by Neo4j; thus, we don't need to convert it from CSV, XML, or some other format.

First, let's download the dataset as a Zip file, unzip it, and add it to the /data folder in our Neo4j directory:

```
$ cd /path/to/neo4j
$ curl -O <copy URL from footnote 4>
$ unzip cineasts_12k_movies_50k_actors_2.1.6.zip
$ mv cineasts_12k_movies_50k_actors.db data/movies.db
```

3. http://neo4j.com/docs/developer-manual/3.0/http-api/#http-api-transactional
4. http://example-data.neo4j.org/files/cineasts_12k_movies_50k_actors_2.1.6.zip

That dataset was generated to work with version 2.1.6 of Neo4j, but we're using a much later version (3.0.7). We'll need to make one small configuration change to enable it to work with our version. In the /conf folder there's a file called neo4j.conf, and inside that file there's a line that looks like this:

```
#dbms.allow_format_migrations=true
```

Delete the # at the beginning of the line. That will instruct Neo4j to automatically migrate the format to fit our version. Now, fire up the Neo4j shell, specifying our movies.db database and the config file we just modified:

```
$ bin/neo4j-shell -path data/movies.db -config conf/neo4j.conf
```

This is our first encounter with the Neo4j shell. It's somewhat like the web console we used earlier in this chapter, but with the crucial difference that it returns raw values rather than pretty charts. It is a more direct and no-frills way to interact with Neo4j and better to use once you have gotten the hang of the database. When the shell fires up, you should see a shell prompt like this:

```
neo4j-sh (?)$
```

You can enter help to get a list of available shell commands. At any time in the shell, you can either enter one of those commands *or* a Cypher query.

Our shell session is already pointing to our movie database, so let's see what nodes are there:

```
neo4j> MATCH (n) RETURN n;
```

Whoa! That's a lot of nodes, 63,042 to be exact (you can obtain that result by returning count(n) instead of just n). We warned you that this is a Big Data section! Let's make some more specific queries now. First, let's see what types of relationships exist:

```
neo4j> MATCH ()-[r]-() RETURN DISTINCT type(r);
+------------+
| type(r)    |
+------------+
| "ACTS_IN"  |
| "DIRECTED" |
| "RATED"    |
| "FRIEND"   |
+------------+
4 rows
```

Here, the ()-[r]-() expresses that we don't care what the nodes look like; we just want to know about the relationship between them, which we're storing in the variable r. You can also see that in Cypher you use type(r) instead of, say,

r.type to get the relationship type (because types are a special attribute of relationships). As you can see, there are four types of relationships present in the database. Now, let's look at all the nodes and see both which labels are applied to them and how many nodes are characterized by that label:

```
$ MATCH (n) RETURN DISTINCT labels(n), count(n);
+----------------------------------------+
| labels(n)                    | count(*) |
+----------------------------------------+
| ["Person","Actor","Director"] | 846      |
| ["Person","User"]             | 45       |
| ["Person","Actor"]            | 44097    |
| ["Person","Director"]         | 5191     |
| []                            | 1        |
| ["Movie"]                     | 12862    |
+----------------------------------------+
6 rows
```

As you can see, all nodes that aren't movies have the Person label (or no label); of those, all Persons are either Actors, Directors, Users (the folks who put the dataset together), or a Director and Actor (we're looking at you, Clint Eastwood). Let's perform some other queries to explore the database.

Let's see everyone who's both an actor and a director, and then get the count of people who share that distinction:

```
> MATCH (p:Actor:Director) RETURN p.name;
> MATCH (p:Actor:Director) RETURN count(p.name);
```

Now let's see who directed the immortal *Top Gun*:

```
> MATCH (d:Director)-[:DIRECTED]-(m:Movie {title: "Top Gun"}) RETURN d.name;
```

Let's see how many movies the legendary Meryl Streep has acted in:

```
> MATCH (a:Actor {name: "Meryl Streep"})-[:ACTS_IN]-(m:Movie)
  RETURN count(m);
```

Finally, let's get a list of actors who have appeared in over fifty movies:

```
> MATCH (a: Actor)-[:ACTS_IN]->(m:Movie)
  WITH a, count(m) AS movie_count
  WHERE movie_count > 50
  RETURN a.name; # only 6 actors!
```

Now that we've played with this specific dataset a little bit, let's solve a more challenging algorithmic problem that uses Neo4j more like the high-powered graph database that it really is. What's the most common algorithmic problem in showbiz? You may have guessed already...

Six Degrees of...

...you guessed it: Kevin Bacon. We're going to solve the six degrees of Kevin Bacon problem here so that you can memorize some of the key results and be a big hit at your next dinner party. More specifically, we want to know how many actors are within six degrees of Mr. Bacon, what percentage of the actors in the database have that distinction, what the shortest "path" from an actor to Kevin Bacon happens to be, and so on. You'll find some similar Neo4j exercises online but this one utilizes a very large dataset that can generate more true-to-life results.

What you may find in this exercise is that Cypher has *a lot* already baked into the language. To get the results we're after, we won't need to write a sophisticated algorithm on the client side or traverse a node tree or anything like that. We just need to learn a little bit more about how Cypher works.

In the last section, you saw that you can make very specific queries about nodes and relationships. Let's find out which `Movies` nodes Kevin Bacon has the relationship `ACTED_IN` with (let's see the number of movies first and then list the titles):

```
> MATCH (Actor {name:"Kevin Bacon"})-[:ACTS_IN]-(m:Movie) RETURN count(m);
> MATCH (Actor {name:"Kevin Bacon"})-[:ACTS_IN]-(m:Movie) RETURN m.title;
```

Only thirty movies in our database! Amazing that such a not-exceedingly prolific actor is so well connected in Hollywood. But remember, the magic of Kevin Bacon is not the *number* of movies he's been in; it's the *variety* of actors he's shared the screen with. Let's find out how many actors share this distinction (this query will make more sense later):

```
> MATCH (Actor {name: "Kevin Bacon"})-[:ACTS_IN]->(Movie)
    <-[:ACTS_IN]-(other:Actor)
  RETURN count(DISTINCT other);
+----------+
| count(a) |
+----------+
| 304      |
+----------+
```

Still not a *huge* number, but remember this is only one degree of Kevin Bacon. Here, we can see that you can actually *reverse* the direction of the relationship arrow in a query, which is quite useful in more complex queries like this.

Now let's find out how many actors are *two* degrees from Kevin Bacon. From the previous expression, it's not entirely clear *how* to write that query because we'd need to make that relationship chain much more complex, resulting in

Be Wary of Repetition

You may have noticed the DISTINCT expression in many of these Cypher queries. This is *extremely* important in Cypher queries because it enables you to exclude redundant results. Running the previous query without using DISTINCT results in a count of 313, which suggests that there are a few actors who are within two degrees of Kevin Bacon more than once. Quite the distinction (no pun intended)!

For some datasets, these discrepancies may be much larger, skewing the results beyond recognition and usefulness. When dealing with graphs, this is something that can really bite you, so if your results ever seem off, checking for redundancy is a good place to start.

a mess of arrows and brackets. Fortunately, Cypher provides us with some syntactical sugar for this using star notation (*).

```
> MATCH (Actor {name: "Kevin Bacon"})-[:ACTS_IN*1..2]-(other:Actor)
  RETURN count(DISTINCT other);
+----------+
| count(a) |
+----------+
| 304      |
+----------+
```

Two things to be aware of. First, there's no Movie label anywhere here. That's because Actors can only have ACTS_IN relationships with Movies, so we can safely leave that part out. Second, note that that's the same result as before (a count of 313), so something is not quite right. It turns out that this Cypher star notation is a bit tricky because each "jump" between nodes counts. So you'll need to think of this query in terms of a four-jump chain (actor-movie-actor-movie-actor) and rewrite the query:

```
> MATCH (Actor {name: "Kevin Bacon"})-[:ACTS_IN*1..4]-(other:Actor)
  RETURN count(DISTINCT other);
+----------------------+
| count(DISTINCT other) |
+----------------------+
| 9096                 |
+----------------------+
```

If you use 5 instead of 4 in that query, you'll get the same result and for the same reason. You need to make an actor-to-movie jump to include more actors in the result set.

As you can see, the quotient between 2 degrees and 1 degree is about 79, so the web of relationships fans out very quickly. Calculating 3 degrees yields 31,323. Counting 4 degrees takes *minutes* and might even time out on your

machine, so we don't recommend running that query unless you need a (long) hot chocolate break.

Thus far, we've only really been counting nodes that share some trait, though our ability to describe those traits has been enhanced. We're still not equipped to answer any of our initial questions, such as how many degrees lie between Kevin Bacon and other actors, what percentage of actors lie within N degrees, and so on.

To get traction into those questions, we need to begin querying for path data, as we did in the REST exercises. Once again, Cypher comes through in the clutch for us with its shortestPath function, which enables you to easily calculate the distance between two nodes. You can specify the relationship type you're interested in specifically or just use * if the relationship type doesn't matter.

We can use the shortestPath function to find the number of degrees separating Kevin Bacon and another dashing actor, Sean Penn, using this query:

```
> MATCH (
    bacon:Actor {name: "Kevin Bacon"}),
    (penn:Actor {name: "Sean Penn"}
  ),
  p=shortestPath((bacon)-[:ACTS_IN*]-(penn))
  RETURN length(p);
+---------------+
| length(p) / 2 |
+---------------+
| 2             |
+---------------+
```

But wait a second. According to IMDB, Messieurs Bacon and Penn starred together in *Mystic River*. So why does it take 2 degrees to connect these two when it should be one? Well, it turns out that *Mystic River* isn't in our database.

```
> MATCH (m:Movie {name: "Mystic River"})
  RETURN count(DISTINCT m);
+-------------------+
| count(DISTINCT m) |
+-------------------+
| 0                 |
+-------------------+
```

Looks like our database is lacking some crucial bits of cinema. So maybe don't use these results to show off at your next dinner party just yet; you might want to find a more complete database for that. But for now, you know how to calculate shortest paths, so that's a good start. Try finding the number of degrees between some of your favorite actors.

Another thing we're interested in beyond the shortest path between any two actors is the percentage of actors that lie within N degrees. We can do that by using a generic `other` node with the `Actor` label and counting the *number* of shortest paths that are found within a number of degrees. We'll start with 2 degrees and divide the result by the total number of actors, making sure to specify that we don't include the original Kevin Bacon node in the shortest path calculation (or we'll get a nasty and long-winded error message).

```
> MATCH p=shortestPath(
    (bacon:Actor {name: "Kevin Bacon"})-[:ACTS_IN*1..2]-(other:Actor)
  )
  WHERE bacon <> other
  RETURN count(p);
+----------+
| count(p) |
+----------+
| 304      |
+----------+
```

Just as expected, the same result as before. What happened there is that Neo4j traversed every relationship within 1 degree of Kevin Bacon and found the number that had shortest paths. So in this case, `p` returns a list of many shortest paths between Kevin Bacon and many actors rather than just a single shortest path to one actor. Now let's divide by the total number of actors (44,943) and add an extra decimal place to make sure we get a float:

```
> MATCH p=shortestPath(
    (bacon:Actor {name: "Kevin Bacon"})-[:ACTS_IN*1..2]-(other:Actor)
  )
  WHERE bacon <> other
  RETURN count(p) / 44943.0;
+----------------------+
| count(p) / 44943.0   |
+----------------------+
| 0.006764123445252876 |
+----------------------+
```

That's a pretty small percentage of actors. But now re-run that query using 4 instead of 2 (to symbolize 2 degrees rather than 1):

```
> MATCH p=shortestPath(
    (bacon:Actor {name: "Kevin Bacon"})-[:ACTS_IN*1..4]-(other:Actor)
  )
  WHERE bacon <> other
  RETURN count(p) / 44943.0;
+--------------------+
| count(p) / 44943.0 |
+--------------------+
| 0.2023674432058385 |
+--------------------+
```

Already up to 20 percent within just 1 extra degree. Running the same query gets you almost 70 percent for 3 degrees, a little over 90 percent for 4 degrees, 93 percent for 5, and about 93.4 percent for a full 6 degrees. So how many actors have *no* relationship with Kevin Bacon whatsoever in our database? We can find that out by not specifying an N for degrees and just using any degree:

```
> MATCH p=shortestPath(
    (bacon:Actor {name: "Kevin Bacon"})-[:ACTS_IN*]-(other:Actor)
  )
  WHERE bacon <> other
  RETURN count(p) / 44943.0;
+--------------------+
| count(p) / 44943.0 |
+--------------------+
| 0.9354960728033287 |
+--------------------+
```

Just a little bit higher than the percentage of actors within 6 degrees, so if you're related to Kevin Bacon *at all* in our database, then you're almost certainly within 6 degrees.

Day 2 Wrap-Up

On Day 2, we broadened our ability to interact with Neo4j by taking a look at the REST interface. You saw how, using the Cypher plugin, you can execute Cypher code on the server and have the REST interface return results. We played around with a larger dataset and finally finished up with a handful of algorithms for diving into that data.

Day 2 Homework

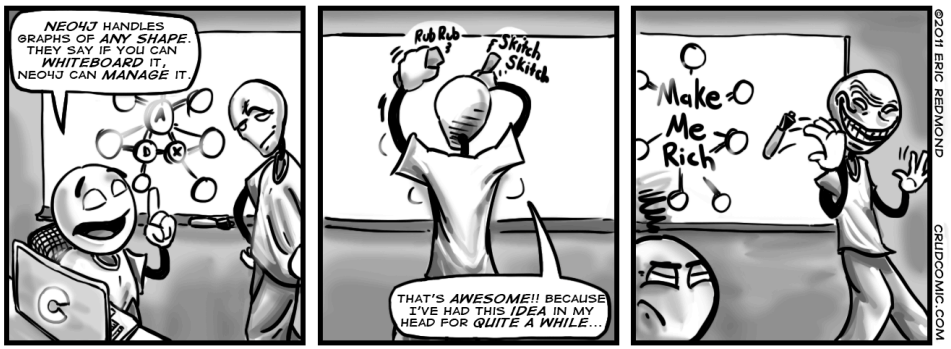
Find

1. Bookmark the documentation for the Neo4j REST API.
2. Bookmark the API for the JUNG project and the algorithms it implements.
3. Find a binding or REST interface for your favorite programming language.

Do

1. Turn the path-finding portion of the Kevin Bacon algorithm into its own step. Then implement a general-purpose Groovy function (for example, `def actor_path(g, name1, name2) {...}`) that accepts the graph and two names and compares the distance.
2. Choose and run one of the many JUNG algorithms on a node (or the dataset, if the API demands it).

3. Install your driver of choice, and use it to manage your company graph with the people and the roles they play, with edges describing their interactions (reports to, works with). If your company is huge, just try your close teams; if you're with a small organization, try including some customers. Find the most well-connected person in the organization by closest distance to all other nodes.



Day 3: Distributed High Availability

Let's wrap up our Neo4j investigation by learning how to make Neo4j more suitable for mission-critical, production uses. We'll see how Neo4j keeps data stable via ACID-compliant transactions. Then we'll install and configure a Neo4j high availability (HA) cluster to improve availability when serving high-read traffic. Then we're going to look into backup strategies to ensure that our data remains safe.

Transactions and Only Transactions with Cypher

Neo4j is an Atomic, Consistent, Isolated, Durable (ACID) transaction database, similar to PostgreSQL. This makes it a good option for important data that you may have otherwise picked a relational database for. Just like transactions you've seen before, Neo4j transactions are all-or-nothing operations. When a transaction starts, every following operation will succeed or fail as an atomic unit—failure of one means failure of all. Much like specifying `BEGIN` and `COMMIT` delimiters as in Postgres, Cypher enables you to do the same thing in the Cypher shell using `:begin` and `:commit` (the debts to the SQL world should be quite clear here!).

If you're using Cypher from a non-shell client, *all* queries are automatically treated as transactions and thus completely succeed or completely fail. Explicit transaction logic is necessary only in the shell.

High Availability

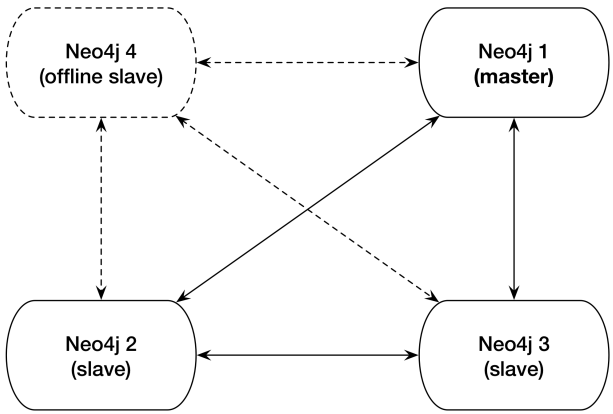
High availability mode is Neo4j's answer to the question, "Can a graph database scale?" Yes, but with some caveats. A write to one slave is not immediately synchronized with all other slaves, so there is a danger of losing consistency (in the CAP sense) for a brief moment (making it eventually consistent). HA will lose pure ACID-compliant transactions. It's for this reason that Neo4j HA is touted as a solution largely for increasing capacity for reads.

Just like Mongo, the servers in the cluster will elect a master that holds primary responsibility for managing data distribution in the cluster. Unlike in Mongo, however, slaves in Neo4j accept writes. Slave writes will synchronize with the master node, which will then propagate those changes to the other slaves.

HA Cluster

To use Neo4j HA, we must first set up a cluster. Previously, Neo4j clusters relied on ZooKeeper as an external coordination mechanism, which worked well but required a lot of additional administration, as ZooKeeper would have to be run separately. That has changed in more recent versions. Now, Neo4j clusters are self-managing and self-coordinating. Clusters can choose their own master/slave setup and re-coordinate when servers go offline.

You can see an illustration of this in the following figure, which shows a 4-node Neo4j cluster.



Nodes 1, 2, and 3 are currently online and replicating to one another properly, while node 4 is offline. When node 4 comes back online, it will re-enter as a slave node. If node 1, the current master node, went offline, the other nodes would automatically elect a leader (without the help of an external coordinating service). This is a fairly standard HA setup in the industry today, and the

engineers behind Neo4j have done administrators a great service by enabling a ZooKeeper-free HA setup.

Building and Starting the Cluster

To build a cluster, we're going to run three instances of Neo4j Enterprise version 3.1.4. You can download a copy from the website for your operating system (be sure you select the correct edition)[5] and then unzip it and create two more copies of the directory. Let's name them neo4j-1.local, neo4j-2.local, and neo4j-3.local.

```
$ tar fx neo4j-enterprise-3.1.4-unix.tar.gz
$ mv neo4j-enterprise-3.1.4 neo4j-1.local
$ cp -R neo4j-1.local neo4j-2.local
$ cp -R neo4j-1.local neo4j-3.local
```

Now we have three identical copies of our database. Normally, you would unpack one copy per server and configure the cluster to be aware of the other servers. In order to build a local cluster, we need to make a few small configuration changes and start the nodes one by one. Each node contains a conf/neo4j.conf configuration file. At the top of that file in the folder for node 1, neo4j-1.local, add this:

```
dbms.mode=HA
dbms.memory.pagecache.size=200m
dbms.backup.address=127.0.0.1:6366
dbms.backup.enabled=true
ha.server_id=1
ha.initial_hosts=127.0.0.1:5001,127.0.0.1:5002,127.0.0.1:5003
ha.host.coordination=127.0.0.1:5001
ha.host.data=127.0.0.1:6363
dbms.connector.http.enabled=true
dbms.connector.http.listen_address=:7474
dbms.connector.bolt.enabled=true
dbms.connector.bolt.tls_level=OPTIONAL
dbms.connector.bolt.listen_address=:7687
dbms.security.auth_enabled=false
```

Copy and paste the same thing into the other two nodes' config files, except increase the following values by 1 *for each node* (producing three separate values for each—for example, 7474, 7475, and 7476 for dbms.connector.http.listen_address):

- ha.server_id
- ha.host.coordination
- ha.host.data
- dbms.connector.http.listen_address
- dbms.connector.bolt.listen_address

5. http://neo4j.org/download/

So, ha.server_id should have values of 1, 2, and 3 on the different nodes, respectively, and so on for the other configs. This is to ensure that the nodes aren't attempting to open up the same ports for the same operations. Now we can start each node one by one (the order doesn't matter):

```
$ neo4j-1.local/bin/neo4j start
$ neo4j-2.local/bin/neo4j start
$ neo4j-3.local/bin/neo4j start
```

You can watch the server output of any of the three running nodes by tailing the log file.

```
$ tail -f neo4j-3.local/logs/neo4j.log
```

If the cluster has been set up successfully, you should see something like this:

```
2017-05-08 03:38:06.901+0000 INFO  Started.
2017-05-08 03:38:07.192+0000 INFO  Mounted REST API at: /db/manage
2017-05-08 03:38:07.902+0000 INFO  Remote interface available at https://...
```

You should also be able to use three different Neo4j browser consoles at http://localhost:7474, as before, but also on ports 7475 and 7476. Don't use the browser console for now, though. We're going to experiment with the cluster via the CLI instead.

Verifying Cluster Status

We now have three different nodes running alongside one another in a cluster, ready to do our bidding. So let's jump straight in and write some data to make sure that things are being properly replicated across the cluster.

Jump into the Cypher shell (as we did on Day 2) for the first node:

```
$ neo4j-1.local/bin/cypher-shell
Connected to Neo4j 3.1.4 at bolt://localhost:7687 as user neo4j.
Type :help for a list of available commands or :exit to exit the shell.
Note that Cypher queries must end with a semicolon.
neo4j>
```

Now let's write a new data node to our cluster and exit the Cypher shell for node 1...

```
neo4j> CREATE (p:Person {name: "Weird Al Yankovic"});
neo4j> :exit
```

...and then open up the shell for node 2...

```
$ neo4j-2.local/bin/cypher-shell -u neo4j -p pass
```

...and finally see which data nodes are stored in the cluster:

```
neo4j> MATCH (n) RETURN n;
(:Person {name: "Weird Al Yankovic"})
```

And there you have it: Our data has been successfully replicated across nodes. You can try the same thing on node 3 if you'd like.

Master Election

In HA Neo4j clusters, master election happens automatically. If the master server goes offline, other servers will notice and elect a leader from among themselves. Starting the previous master server again will add it back to the cluster, but now the old master will remain a slave (until another server goes down).

High availability allows very read-heavy systems to deal with replicating a graph across multiple servers and thus sharing the load. Although the cluster as a whole is only eventually consistent, there are tricks you can apply to reduce the chance of reading stale data in your own applications, such as assigning a session to one server. With the right tools, planning, and a good setup, you can build a graph database large enough to handle billions of nodes and edges and nearly any number of requests you may need. Just add regular backups, and you have the recipe for a solid production system.

Backups

Backups are a necessary aspect of any professional database use. Although backups are effectively built in when using replication in a highly available cluster, periodic backups—nightly, weekly, hourly, and so on—that are stored off-site are always a good idea for disaster recovery. It's hard to plan for a server room fire or an earthquake shaking a building to rubble.

Neo4j Enterprise offers a tool called `neo4j-admin` that performs a wide variety of actions, including backups.

The most powerful method when running an HA server is to craft a full backup command to copy the database file from the cluster to a date-stamped file on a mounted drive. Pointing the copy to every server in the cluster will ensure you get the most recent data available. The backup directory created is a fully usable copy. If you need to recover, just replace each installation's data directory with the backup directory, and you're ready to go.

You must start with a full backup. Let's back up our HA cluster to a directory that ends with today's date (using the *nix date command). The `neo4j-admin` command can be run from any server and you can choose any server in the cluster when using the `--from` flag. Here's an example command:

```
$ neo4j-1.local/bin/neo4j-admin backup \
  --from 127.0.0.1:6366 \
  --name neo4j-`date +%Y.%m.%d`.db \
  --backup-dir /mnt/backups
```

Once you have done a full backup, you can choose to do an incremental backup by specifying an existing `.db` database directory as the target directory. But keep in mind that incremental backups only work on a fully backed-up directory, so ensure the previous command is run on the same day or the directory names won't match up.

Day 3 Wrap-Up

Today we spent some time keeping Neo4j data stable via ACID-compliant transactions, high availability, and backup tools.

It's important to note that all of the tools we used today require the Neo4j Enterprise edition, and so use a dual license—GPL/AGPL. If you want to keep your server closed source, you should look into switching to the Community edition or getting an OEM from Neo Technology (the company behind Neo4j). Contact the Neo4j team for more information.

Day 3 Homework

Find

1. Find the Neo4j licensing guide.
2. Answer the question, "What is the maximum number of nodes supported?" (Hint: It's in Questions & Answers in the website docs.)

Do

1. Replicate Neo4j across three physical servers.
2. Set up a load balancer using a web server such as Apache or Nginx, and connect to the cluster using the REST interface. Execute a Cypher script command.
3. Experiment further with the `neo4j-admin` tool. Acquire a solid understanding of *three* subcommands beyond `backup`.

Wrap-Up

Neo4j is a top open source implementation of the (relatively rare) class of graph databases. Graph databases focus on the relationships between data, rather than the commonalities among values. Modeling graph data is simple.

You just create nodes and relationships between them and optionally hang key-value pairs from them. Querying is as easy as declaring how to walk the graph from a starting node.

Neo4j's Strengths

Neo4j is one of the finest examples of open source graph databases. Graph databases are perfect for unstructured data, in many ways even more so than document databases. Not only is Neo4j typeless and schemaless, but it puts no constraints on how data is related. It is, in the best sense, a free-for-all. Currently, Neo4j can support 34.4 billion nodes and 34.4 billion relationships, which is more than enough for most use cases. (Neo4j could hold more than 15 nodes for each of Facebook's 2.2 billion users in a single graph.)

The Neo4j distributions provide several tools for fast lookups with Lucene, the Cypher querying language, and the REST interface. Beyond ease of use, Neo4j is fast. Unlike join operations in relational databases or map-reduce operations in other databases, graph traversals are constant time. Like data is only a node step away, rather than joining values in bulk and filtering the desired results, which is how most of the databases we've seen operate. It doesn't matter how large the graph becomes; moving from node A to node B is always one step if they share a relationship. Finally, the Enterprise edition provides for highly available and high read-traffic sites by way of Neo4j HA.

Neo4j's Weaknesses

Neo4j does have a few shortcomings. We found its choice of nomenclature (*node* rather than *vertex* and *relationship* rather than *edge*) to add complexity when communicating. Although HA is excellent at replication, it can only replicate a full graph to other servers. It cannot currently shard subgraphs, which still places a limit on graph size (though, to be fair, that limit measures in the tens of billions). Finally, if you are looking for a business-friendly open source license (like MIT), Neo4j may not be for you. Although the Community edition (everything we used in the first two days) is GPL, you'll probably need to purchase a license if you want to run a production environment using the Enterprise tools (which includes HA and backups).

Neo4j on CAP

The term "high availability cluster" should be enough to give away Neo4j's strategy. Neo4j HA is available and partition tolerant (AP). Each slave will return only what it currently has, which may be out of sync with the master node temporarily. Although you can reduce the update latency by increasing

a slave's pull interval, it's still technically eventually consistent. This is why Neo4j HA is recommended for read-mostly requirements.

Parting Thoughts

Neo4j's simplicity can be off-putting if you're not used to modeling graph data. It provides a powerful open-source API with years of production use and yet it hasn't gotten the same traction as other databases in this book. We chalk this up to lack of knowledge because graph databases mesh so naturally with how humans tend to conceptualize data. We imagine our families as trees, or our friends as graphs; most of us don't imagine personal relationships as self-referential datatypes. For certain classes of problems, such as social networks, Neo4j is an obvious choice. But you should give it some serious consideration for non-obvious problems as well—it just may surprise you how powerful and easy it is.